

BadSQL

Design Document

v0.1.0-alpha · C++23 · TCP Server Architecture

Autore: RickIsGone

Repository: github.com/RickIsGone/BadSQL

Linguaggio implementazione: C++23 + Boost.Asio

Stato: Alpha 0.1.0 · 2026

1. Overview del Progetto

BadSQL è un database server scritto da zero in C++23 con architettura client-server su TCP/IP. Il server accetta connessioni sulla porta 5432, tokenizza query in formato testo e le elabora tramite un lexer avanzato con supporto per position tracking e error reporting accurato. Utilizza Boost.Asio per la gestione asincrona delle connessioni e un sistema di logging colorizzato con supporti per thread safety.

1.1 Stack Tecnologico Reale

Componente	Tecnologia	Descrizione
Linguaggio	C++23	std::expected, std::optional, std::format
Rete	Boost.Asio	TCP async, signal handling, thread pool
Build	CMake 3.30+	Cross-platform compilation
Logging	Nativo C++23	std::syncstream, colorized output
Server Port	TCP 5432	PostgreSQL port (compatibility)

1.2 Componenti Implementati vs0.1.0)

Componente	Status	Dettagli
Token System	✓ Completato	44 token types, Position tracking, Lexeme storage
Lexer	✓ Completato	Tokenizzazione avanzata, String handling, Numeric literals
Logger	✓ Completato	4 livelli (Errors/Warnings/Info/Debug), Timestamp, Colors, Thread-safe
Query Engine (Handler)	✓ Completato	TCP socket management, Query reception, Lexer integration
TCP Server	✓ Completato	Boost.Asio async, Multi-connection, Signal handling (SIGINT/SIGTERM)
Parser	📋 Non iniziato	AST construction — prossimo step
Executor	📋 Non iniziato	Query execution — dopo parser
Storage	📋 Non iniziato	In-memory catalog — dopo executor

2. Token System — Implementazione Reale (token.h)

2.1 TokenType Enum Completo

44 token types organizzati per categoria semantica:

```
enum class TokenType {
    Invalid,

    // DDL Keywords (6)
    Create, Drop, Delete, Insert, Update,
    Database, Table,

    // DML Keywords (8)
    Use, From, Where, Into, Values, All,

    // Literals (4)
    Identifier, IntLiteral, FloatLiteral, String,

    // Punctuation (4)
    OpenParen, CloseParen, Comma, SemiColon,

    // Type Keywords (6)
    I32, U32, F32, F64, Varchar, Char,

    // Operators (6)
    Greater, GreaterEqual, Less, LessEqual, Equal, NotEqual
};
```

2.2 Position Struct

```
struct Position {
    size_t line;
    size_t column;
};
```

2.3 Token Struct

```
struct Token {
    TokenType type;           // Tipo enum del token
    Position position;        // Line + column per error reporting
    std::string lexeme;       // Testo raw del token
};
```

2.4 Keyword Mapping — unordered_map

```
static std::unordered_map<std::string_view, TokenType> tokenMap = {
    {"CREATE", TokenType::Create},
    {"DROP", TokenType::Drop},
    {"DELETE", TokenType::Delete},
    {"INSERT", TokenType::Insert},
    {"UPDATE", TokenType::Update},
    {"DATABASE", TokenType::Database},
    {"TABLE", TokenType::Table},
    {"USE", TokenType::Use},
    {"FROM", TokenType::From},
    {"WHERE", TokenType::Where},
    {"INTO", TokenType::Into},
    {"VALUES", TokenType::Values},
    {"i32", TokenType::I32},
    {"u32", TokenType::U32},
    {"f32", TokenType::F32},
    {"f64", TokenType::F64},
    {"varchar", TokenType::Varchar},
    {"char", TokenType::Char}
};
```

2.5 Token Matching Function

```
inline std::optional<TokenType> tryMatchToken(std::string_view token) {
    if (tokenMap.contains(token))
        return tokenMap.at(token);
    return std::nullopt;
}
```

3. Lexer — Implementazione Reale (lexer.h/cpp)

3.1 Classe Lexer

```
class Lexer {
public:
    Lexer(std::string_view query) : _query(query) {}
    ~Lexer() = default;
    Lexer(const Lexer&) = delete;           // No copy
    Lexer& operator=(const Lexer&) = delete;
    Lexer(Lexer&&) = delete;               // No move

    [[nodiscard]] std::expected<std::vector<Token>, std::string> tokenize();

private:
    [[nodiscard]] std::optional<char> _peek(const size_t offset = 0) const;
    [[nodiscard]] std::optional<char> _peekBack(const size_t offset = 0) const;
    [[nodiscard]] std::optional<TokenType> _determineFixedSizeTokenType(
        std::string_view token) const;
    [[nodiscard]] Position _position(const size_t index) const;
    void _consume(size_t amount = 1);

    size_t _index{0};
    std::string_view _query;
};
```

3.2 Algoritmo di Tokenizzazione

Il metodo tokenize() segue questa logica:

1. Salta whitespace (spazi, tab, newline)
2. Se carattere alfabetico o '_':
 - Accumula alfanumerici
 - Converte a maiuscoli
 - Lookup nella tokenMap
 - Crea token (keyword o identifier)
3. Se cifra:
 - Accumula cifre e punto
 - Crea token IntLiteral o FloatLiteral
4. Se apice '
 - Accumula fino al prossimo apice
 - Supporta '' (escape sequence)
 - Errore se newline non closed
5. Se operatore (>, <, !, =, etc):
 - Usa _determineFixedSizeTokenType()
 - Crea token per >, >=, <, <=, =, !=
6. Se simbolo () , ; * :
 - Crea token direttamente
7. Altrimenti: errore UNEXPECTED_TOKEN

3.3 Error Handling Dettagliato

std::expected<std::vector<Token>, std::string> ritorna:

- ✗ "Unterminated string literal at line 5 col 12"
 - ✗ "Newline in string literal at line 3 col 8"
 - ✗ "Unexpected token starting at line 1 col 15: 'xyz'"
- ✓ Vector<Token> con Position tracking per ogni token

4. Logger — Sistema di Logging Avanzato (logger.h)

4.1 Caratteristiche

- 4 livelli: Errors (rosso), Warnings (giallo), Info (verde), Debug (blu)
- Timestamp con millisecond precision
- Thread-safe via std::osyncstream (C++20)
- ANSI color codes built-in
- std::format per formatted output

4.2 Enum LogLevel

```
enum class LogLevel {  
    None,          // Nessun output  
    Errors,        // Solo errori (default)  
    Warnings,      // Errori + avvisi  
    Info,          // Errori + avvisi + info  
    Debug         // Tutto  
};
```

4.3 API di Logging

```
Logger::setLogLevel(LogLevel::Debug);  
  
Logger::error("Connection failed: {}", e.what());  
Logger::warn("Slow query: {} ms", duration);  
Logger::info("Server listening on port {}", PORT);  
Logger::debug("Token: {} at {},{}", token.type, line, col);
```

5. Query Engine — Handler e TCP (queryEngine.h/cpp)

5.1 Funzione handleQuery()

```
void handleQuery(tcp::socket socket) {
    std::string clientInfo = "<unknown>";
    try {
        // 1. Estrai endpoint client
        auto ep = socket.remote_endpoint();
        clientInfo = ep.address().to_string() + ":" + port;

        // 2. Ricevi query (max 8KB)
        std::array<char, 8192> buffer{};
        size_t bytes = socket.read_some(boost::asio::buffer(buffer));
        std::string query(buffer.data(), bytes);

        // 3. Tokenizza
        Lexer lexer{query};
        auto res = lexer.tokenize();
        if (!res) throw std::runtime_error{res.error()};

        // 4. Ritorna OK
        std::string result = "OK";
        boost::asio::write(socket, boost::asio::buffer(result));
        Logger::info("{} -- {}", clientInfo, result);
    } catch (std::exception& e) {
        Logger::error("{} -- {}", clientInfo, e.what());
        boost::asio::write(socket, boost::asio::buffer(e.what()));
    }
    socket.close();
}
```


6. TCP Server — Implementazione con Boost.Asio (main.cpp)

6.1 Server Loop Asincrono

```
// Setup ASIO context
boost::asio::io_context io;
tcp::acceptor acceptor(io, tcp::endpoint(tcp::v4(), PORT));
acceptor.set_option(boost::asio::socket_base::reuse_address(true));

// Signal handling (SIGINT, SIGTERM)
boost::asio::signal_set signals(io, SIGINT, SIGTERM);
signals.async_wait([&](const boost::system::error_code&, int) {
    Logger::info("Shutdown signal received...");
    io.stop();
});

// Accept connections
std::function<void()> accept = [&]() {
    auto socket = std::make_shared<tcp::socket>(io);
    acceptor.async_accept(*socket, [&, socket](const boost::system::error_code&
ec) {
        if (!ec) {
            threadsHandler.emplace_back([sock = std::move(*socket)]() mutable {
                handleQuery(std::move(sock));
            });
        }
        if (!io.stopped()) accept(); // Loop
    });
};

accept();
io.run(); // Event loop
```

6.2 Flusso di Esecuzione

1. Server starts su porta 5432
2. Listener accetta connessioni async
3. Per ogni client:
 - Crea nuovo thread
 - Chiama handleQuery()
 - Tokenizza query via Lexer
 - Ritorna OK o errore
 - Chiude socket
4. SIGINT/SIGTERM triggera graceful shutdown
5. io_context attende thread completion

7. Tipo di Dati Supportati

Tipo BadSQL	TokenType	Descrizione
i32	I32	Intero con segno 32-bit
u32	U32	Intero senza segno 32-bit
f32	F32	Floating point 32-bit (IEEE 754)
f64	F64	Double precision 64-bit
varchar	Varchar	Stringa con lunghezza max
char	Char	Carattere singolo

Operatori Supportati

Operatore	TokenType	Uso
=	Equal	Comparazione o assegnamento
!=	NotEqual	Diverso da
>	Greater	Maggiore di
>=	GreaterEqual	Maggiore o uguale
<	Less	Minore di
<=	LessEqual	Minore o uguale

8. Roadmap v0.2+

8.1 v0.2 — Parser e AST

- Costruire Abstract Syntax Tree da token stream
- Implementare nodi AST: CreateDatabaseStmt, CreateTableStmt, SelectStmt, etc
- Error recovery nel parser
- Supporto WHERE, ORDER BY, LIMIT (nel parser)

8.2 v0.3 — Executor

- Visita AST e delega al catalogo
- Operazioni CRUD: CREATE, INSERT, SELECT, UPDATE, DELETE
- Risposta strutturata (tabulare)

8.3 v0.4 — Storage Persistence

- In-memory catalog → Disk pages
- Write-Ahead Log (WAL)
- Recovery al boot

8.4 v0.5+ — Indici e Optimizzazioni

- B-Tree indexes
- Query optimizer
- Transactions (ACID)

9. Struttura File Repository

9.1 Layout Attuale

```
BadSQL/
├─ CMakeLists.txt
├─ README.md
├─ test.badsql
├─ client.py          ← Temporaneo (test client)
├─ docs/
│   └─ BadSQL design.pdf
├─ src/
│   ├── token.h        ✓ 44 token types
│   ├── lexer.h        ✓ Classe Lexer
│   ├── lexer.cpp      ✓ Implementazione
│   ├── logger.h       ✓ Sistema logging
│   ├── queryEngine.h  ✓ Handler TCP
│   ├── queryEngine.cpp ✓ Implementazione
│   └─ main.cpp        ✓ Server Boost.Asio
```

9.2 Prossimi File

```
├─ src/
│   ├── ast.hpp        (parser nodes)
│   ├── parser.hpp / .cpp (AST construction)
│   ├── executor.hpp / .cpp
│   ├── catalog.hpp
│   └─ server.hpp / .cpp (refactor from main)
```

10. Compilazione e Utilizzo

10.1 Build

```
git clone https://github.com/RickIsGone/BadSQL.git
cd BadSQL
cmake -B build -DCMAKE_BUILD_TYPE=Release
cmake --build build -j$(nproc)
./build/BadSQL
```

10.2 Test (Temporaneo)

```
# Terminal 1: Server
./build/BadSQL

# Terminal 2: Client
python3 client.py
```

10.3 Test Manuale

```
echo 'SELECT * FROM users;' | nc -q 1 127.0.0.1 5432
```

11. Glossario

Termine	Significato
AST	Abstract Syntax Tree — struttura logica della query
Lexeme	Testo raw del token (il valore effettivo)
Lexer	Tokenizzatore — text → tokens
Parser	Analizzatore sintattico — tokens → AST
Position	Line + column per error reporting
std::expected	Tipo per risultati con error info
std::optional	Tipo per valori che possono mancare
Token	Unità elementare (type + position + lexeme)
TokenType	Enumerazione dei 44 tipi di token